```
"""Телеграм-бот поиска музыки.
Ищет через Deezer API (бесплатно, без ключа). Если заданы SPOTIFY_CLIENT_ID/SECRE
Присылает: название, исполнителя, альбом, превью 30 с и ссылки (Deezer, Spotify,
import hashlib
import html
import logging
import os
import time
from urllib.parse import quote

import httpx
from telegram import InlineKeyboardButton, InlineKeyboardMarkup, Update
from telegram.ext import (Application, CallbackQueryHandler, CommandHandler,
                          ContextTypes, MessageHandler, filters)

logging.basicConfig(level=logging.INFO)
TOKEN = os.environ["BOT_TOKEN"]                         # токен от @BotFather
SP_ID = os.environ.get("SPOTIFY_CLIENT_ID")
SP_SECRET = os.environ.get("SPOTIFY_CLIENT_SECRET")
http = httpx.AsyncClient(timeout=15)
_sp = {"token": None, "exp": 0.0}


async def deezer_search(q: str, limit: int = 5) -> list[dict]:
    r = await http.get("https://api.deezer.com/search", params={"q": q, "limit":
    r.raise_for_status()
    return r.json().get("data", [])


async def spotify_link(artist: str, title: str) -> str | None:
    if not (SP_ID and SP_SECRET):
        return None
    try:
        if time.time() > _sp["exp"]:
            r = await http.post("https://accounts.spotify.com/api/token",
                                data={"grant_type": "client_credentials"}, auth=(
            r.raise_for_status()
            _sp.update(token=r.json()["access_token"], exp=time.time() + r.json()
        r = await http.get("https://api.spotify.com/v1/search",
                           params={"q": f"{artist} {title}", "type": "track", "li
                           headers={"Authorization": f"Bearer {_sp['token']}"})
        items = r.json().get("tracks", {}).get("items", [])
        return items[0]["external_urls"]["spotify"] if items else None
    except Exception:
```

```
        logging.exception("Spotify")
        return None


async def start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Привет! Напиши название песни или «исполните


async def search(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = update.message.text.strip()
    try:
        tracks = await deezer_search(q)
    except Exception:
        logging.exception("Deezer")
        await update.message.reply_text("Поиск сейчас недоступен, попробуйте позж
        return
    if not tracks:
        await update.message.reply_text("Ничего не нашёл. Попробуйте «исполнитель
        return
    for t in tracks:
        artist, title = t["artist"]["name"], t["title"]
        query = quote(f"{artist} {title}")
        sp = await spotify_link(artist, title)
        links = [f'<a href="{t["link"]}">Deezer</a>']
        links.append(f'<a href="{sp}">Spotify</a>' if sp else f'<a href="https://
        links.append(f'<a href="https://music.yandex.ru/search?text={query}">Янде
        links.append(f'<a href="https://www.youtube.com/results?search_query={que
        text = (f"🎵 <b>{html.escape(title)}</b>\n👤 {html.escape(artist)}\n"
                f"💿 {html.escape(t['album']['title'])}\n\n" + " · ".join(links))
        kb = InlineKeyboardMarkup([[InlineKeyboardButton("▶️ Превью 30 с", callba
        await update.message.reply_text(text, parse_mode="HTML", reply_markup=kb,


async def preview(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    try:
        r = await http.get(f"https://api.deezer.com/track/{q.data.split(':')[1]}"
        t = r.json()
        if not t.get("preview"):
            await q.message.reply_text("Для этого трека нет превью.")
            return
        await q.message.reply_audio(t["preview"], title=t["title"], performer=t["
    except Exception:
        logging.exception("preview")
        await q.message.reply_text("Не удалось отправить превью.")
```

```
def main():
    app = Application.builder().token(TOKEN).build()
    app.add_handler(CommandHandler("start", start))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, search))
    app.add_handler(CallbackQueryHandler(preview, pattern=r"^pv:\d+$"))
    base_url = os.environ.get("RENDER_EXTERNAL_URL")      # Render задаёт сам
    if base_url:
        # режим вебхука: Telegram сам обращается к боту (будит бесплатный сервис
        path = hashlib.sha256(TOKEN.encode()).hexdigest()[:32]
        app.run_webhook(listen="0.0.0.0", port=int(os.environ.get("PORT", 10000))
                        url_path=path, webhook_url=f"{base_url}/{path}",
                        secret_token=path, drop_pending_updates=False)
    else:
        app.run_polling()                                  # локальный запуск


if __name__ == "__main__":
    main()
```